2. Text to speech (TTS, online/offline)

2. Text to speech (TTS, online/offline)

1. Concept introduction
   1.1 What is "TTS"?
   1.2 Overview of the implementation principle
      1. **Text analysis**
      2. **Language processing**
      3. **Speech synthesis**
      4. **Sound waveform generation**
2. Code analysis
   Key code
      1. TTS initialization and invocation (`largemodel/largemodel/model_service.py`)
      2. TTS backend implementation (`largemodel/utils/large_model_interface.py`)
   Code analysis
3. Hands-on operation
   3.1 Configure the TTS
      3.1.1 Configure the offline TTS
      3.1.2 Configure the online TTS
   3.2 Start it and test the function
4. Common problems and solutions
   4.1 Playback problems
      Problem 1: The program runs normally without any error, but no sound is heard.

---

1. Concept introduction

1.1 What is "TTS"?

TTS technology converts written text into human-listenable speech output. It enables computers to "read" text aloud and is widely used in a variety of fields, including accessible reading, intelligent assistants, navigation systems, educational software and so on. Through TTS, users can hear natural and fluent machine-generated human voices, which greatly improves the convenience and flexibility of information acquisition.

1.2 Overview of the implementation principle

The implementation of a TTS system mainly involves the following key steps and technologies:

1. Text analysis

- In this stage, the input text is first preprocessed, including but not limited to removing irrelevant characters, standardizing punctuation and capitalization, segmenting words, and recognizing special symbols such as numbers and converting them into their corresponding word forms.
- At the same time, linguistic analysis of the text is also needed, such as determining the pronunciation of each word (this usually requires a pronunciation dictionary), the stress position, the intonation pattern and the sentence structure.

2. Language processing

- This step focuses on how to pronounce and adjust the intonation correctly according to the context. For example, the word "read" has different pronunciations in different tenses (the past tense/past participle is pronounced as /red/, while in other cases it is pronounced as /riːd/). Therefore, a powerful language model is needed to understand these subtle differences.
- Prosody modeling is also involved here, that is, deciding which parts should be emphasized, whether the speaking rate should be fast or slow, and the emotional color of the whole sentence.

3. Speech synthesis

- After being processed by the previous two stages, the text information is fed into the speech synthesis engine, which is responsible for generating the actual sound waveform.
- Traditional TTS systems use concatenative synthesis, that is, selecting suitable units from a database of pre-recorded speech segments and concatenating them to form a complete sentence. Although this method can produce high-quality speech, it is limited by the samples in the database.
- Modern TTS systems rely more on parametric synthesis or neural network synthesis (such as WaveNet, Tacotron, etc.). These methods can directly predict speech features from text and generate continuous sound signals. In particular, the deep learning based methods can capture the subtle variations of speech better, and thus produce more natural and fluent sound.

4. Sound waveform generation

- Finally, the generated sound waveform is further processed to make sure that its quality meets the expected standard, for example by adjusting the volume and equalizing the frequency response.
- After that, this audio data can be played back through a speaker or other audio playback device for people to listen to.

With the development of artificial intelligence and machine learning technologies, especially the application of deep learning, TTS systems have not only improved significantly in accuracy, but also made great progress in naturalness and emotional expression, which makes the machine-generated voice increasingly close to a real human voice.

2. Code analysis

Key code

1. TTS initialization and invocation (`largemodel/largemodel/model_service.py`)

```
# From largemodel/largemodel/model_service.py
class LargeModelService(Node):
    def __init__(self):
        # ...
        self.system_sound_init()
        # ...

    def init_param_config(self):
        # ...
        self.declare_parameter('useolinetts', False)
```

```
        self.useolinetts =
self.get_parameter('useolinetts').get_parameter_value().bool_value
        if self.useolinetts:
            self.tts_out_path = os.path.join(self.pkg_path, "resources_file",
"tts_output.mp3")
        else:
            self.tts_out_path = os.path.join(self.pkg_path, "resources_file",
"tts_output.wav")

    def system_sound_init(self):
        """Initialize TTS system"""
        model_type = "oline" if self.useolinetts else "local"
        self.model_client.tts_model_init(model_type, self.language)
        self.get_logger().info(f'TTS initialized with {model_type} model')

    def _safe_play_audio(self, text_to_speak: str):
        """
        Synthesizes and plays all non-empty messages only in non-text chat mode.
        """
        if not self.text_chat_mode and text_to_speak:
            try:
                self.model_client.voice_synthesis(text_to_speak, self.tts_out_path)
                self.play_audio_async(self.tts_out_path)
            except Exception as e:
                self.get_logger().error(f"Safe audio playback failed: {e}")
```

2. TTS backend implementation (`largemodel/utils/large_model_interface.py`)

```
# From largemodel/utils/large_model_interface.py
class model_interface:
    # ...
    def tts_model_init(self,model_type='oline',language='zh'):
        if model_type=='oline':
            if self.tts_supplier=='baidu':
                self.token=self.fetch_token()

            self.model_type='oline'
        elif model_type=='local':
            self.model_type='local'
            if language=='zh':
                tts_model=self.zh_tts_model
                tts_json=self.zh_tts_json
            elif language=='en':
                tts_model=self.en_tts_model
                tts_json=self.en_tts_json
            self.synthesizer = piper.PiperVoice.load(tts_model, config_path=tts_json,
use_cuda=False)

    def voice_synthesis(self,text,path):
        if self.model_type=='oline':
            if self.tts_supplier=='baidu':
```

```
                # ... (Baidu TTS implementation)
                pass
            elif self.tts_supplier=='aliyun':
                # ... (Aliyun TTS implementation)
                pass
        elif self.model_type=='local':
            with wave.open(path, 'wb') as wav_file:
                wav_file.setnchannels(1)
                wav_file.setsampwidth(2)
                wav_file.setframerate(self.synthesizer.config.sample_rate)
                self.synthesizer.synthesize(text, wav_file)
```

Code analysis

The text to speech (TTS) function is invoked by the `LargeModelService` node and is implemented by the `model_interface` class. Its design switches between different backend services through the parameter configuration.

1. **Initialization flow (`model_service.py`)**:
    - During the initialization of `LargeModelService`, the `init_param_config` function reads the boolean value `useolinetts` from the ROS parameter server.
    - The `system_sound_init` function decides, according to the value of `useolinetts`, whether to pass the string `'local'` or `'oline'` to the `self.model_client.tts_model_init` method.
    - In `large_model_interface.py`, the `tts_model_init` method executes the corresponding initialization logic according to the received string parameter. If it is `'local'`, `piper.PiperVoice.load` is used to load the local model file.
2. **Synthesis and playback flow (`model_service.py`)**:
    - When voice broadcast is needed, the `_safe_play_audio` function is called.
    - This function first calls the `self.model_client.voice_synthesis` method, passing in the text to be converted and the target audio path `self.tts_out_path`.
    - After the `voice_synthesis` method has finished and generated the audio file, `_safe_play_audio` calls `self.play_audio_async` to play the file asynchronously.
3. **Backend implementation selection (`large_model_interface.py`)**:
    - The `voice_synthesis` method is the backend dispatch center of the TTS function. Internally it selects the execution path by checking the value of the `self.model_type` attribute set during initialization.
    - If `self.model_type` is `'local'`, the code block uses the Python `wave` library to open a WAV file, sets its header parameters (channels, sample bit width, sample rate), and then calls the `self.synthesizer.synthesize` method to write the synthesized audio stream of the text directly into that file.
    - If `self.model_type` is `'oline'`, it enters the logic branch for the different cloud service providers (such as Baidu, Aliyun).
    - This structure separates the call of the upper node ("speak this passage") from the concrete synthesis technology at the lower level (which library to use, which API to call).

3. Hands-on operation

3.1 Configure the TTS

The online and the offline TTS share the language, the speaker and the audio device settings, and the synthesis backend is selected with `useolinetts`. First open the main configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

Set the language and the backend in the existing `model_service.ros__parameters` block:

```
model_service:
  ros__parameters:
    language: 'en'              # 'zh': Chinese; 'en': English
    useolinetts: False          # False: offline TTS; True: online TTS
    llm_platform: 'ollama'      # Optional platforms: 'ollama', 'tongyi', 'spark', 'qianfan'
    regional_setting: "International"
```

If the parameters already exist, simply modify the original values and do not add duplicate keys in the same YAML parameter block.

3.1.1 Configure the offline TTS

Set `useolinetts` to `False`, and confirm in `large_model_interface.yaml` that the paths of the Piper model and the configuration file of the selected language exist:

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml::
```

```
zh_tts_model: "/home/jetson/yahboom_ws/src/largemodel/MODELS/tts/zh/zh_CN-huayan-
medium.onnx"
zh_tts_json: "/home/jetson/yahboom_ws/src/largemodel/MODELS/tts/zh/zh_CN-huayan-
medium.onnx.json"
en_tts_model: "/home/jetson/yahboom_ws/src/largemodel/MODELS/tts/en/en_US-libritts-
high.onnx"
en_tts_json: "/home/jetson/yahboom_ws/src/largemodel/MODELS/tts/en/en_US-libritts-
high.onnx.json"
```

3.1.2 Configure the online TTS

The online TTS requires the API credentials of the corresponding service provider. The source tutorial takes Baidu Intelligent Cloud as an example; fill in your own credentials and voice parameters in `large_model_interface.yaml`:

```
# large_model_interface.yaml

# Baidu Intelligent Cloud speech synthesis configuration
baidu_API_KEY: 'your api key'        # Baidu platform API key
baidu_SECRET_KEY: 'your secret key'  # Baidu platform SECRET key
CUID: 'your device identifier '         # Device identifier
PER: 4  # Speaker: basic voice library (0=Du Xiaomei, 1=Du Xiaoyu, 3=Du Xiaoyao, 4=Du Yaya)
SPD: 5  # Speech rate (0-15), default 5
PIT: 5  # Pitch (0-15), default 5
VOL: 5  # Volume (0-9), default 5
```

Make sure `model_service.ros__parameters.useolinetts` in `yahboom.yaml` is `True`, and set the online service provider according to the current `tts_supplier` of the project. The online synthesis requires the Jetson to be able to reach the service provider interface; do not publish a real key.

3.2 Start it and test the function

1. **Start the TTS node**:
   Run the following command:

   ```
   ros2 launch largemodel tts_only.launch.py
   ```

   ```
   :~/yahboom_ws$ ros2 launch largemodel tts_only.launch.py
   [INFO] [launch]: All log files can be found below /home/sunrise/.ros/log/2025-08-07-16-44-02-726089-ubuntu-33939
   [INFO] [launch]: Default logging verbosity is set to INFO
   [INFO] [tts_only-1]: process started with pid [33940]
   [tts_only-1] [INFO] [1754556253.838432585] [tts_only_node]: TTS Only Node is starting...
   [tts_only-1] [INFO] [1754556253.840804978] [tts_only_node]: Language set to: zh
   [tts_only-1] [INFO] [1754556253.841469608] [tts_only_node]: Using online TTS: False
   [tts_only-1] [INFO] [1754556253.842111438] [tts_only_node]: TTS output path: /home/sunrise/yahboom_ws/install/largemodel
   /share/largemodel/resources_file/tts_output.wav
   [tts_only-1] [INFO] [1754556255.723894754] [tts_only_node]: TTS initialized with local model
   ```

2. **Send the text to be synthesized**:
   Open a new terminal and run the following command to publish a voice broadcast message:

   ```
   ros2 topic pub --once /tts_text_input std_msgs/msg/String '{data: "Speech synthesis
   test successful"}'
   ```

3. **Test**:
   If everything is fine, you should hear the robot say "Speech synthesis test successful" with the synthesized voice from your speaker.

---

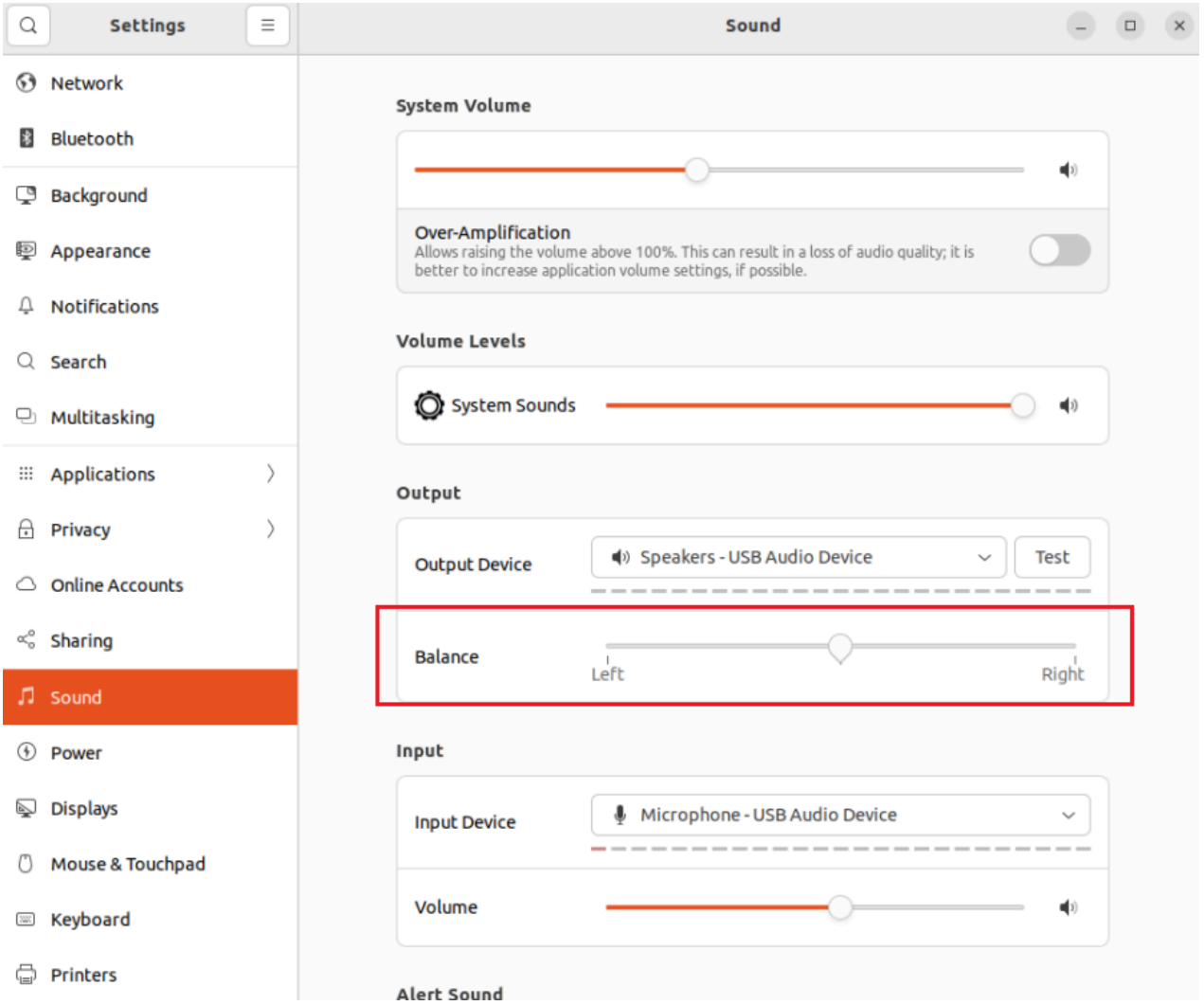
4. Common problems and solutions

4.1 Playback problems

Problem 1: The program runs normally without any error, but no sound is heard.

Solution:

1. **Check the audio output**: Confirm that the audio output device of your system is selected correctly and that the volume is not muted. You can try playing an ordinary music file to test the hardware.
2. In the Sound settings, keep the Balance in the middle position


Settings
Sound
Network
Bluetooth
Background
Appearance
Notifications
Search
Multitasking
Applications
Privacy
Online Accounts
Sharing
Sound
Power
Displays
Mouse & Touchpad
Keyboard
Printers
System Volume
Over-Amplification
Allows raising the volume above 100%. This can result in a loss of audio quality; it is better to increase application volume settings, if possible.
Volume Levels
System Sounds
Output
Output Device
Speakers - USB Audio Device
Test
Balance
Left
Right
Input
Input Device
Microphone - USB Audio Device
Volume
Alert Sound